CJ Nicko B. Rosales
C3A

SOURCE CODE IN PYTHON:

```
"""
MACHINE PROBLEM #1
FCFS (First Come First Serve) CPU Scheduling Simulator

Computes Turnaround Time and Waiting Time for a set of processes
using the non-preemptive FCFS scheduling algorithm.

Validation rules (per spec):
- Number of processes: 3 - 10 only
- Process ID: must be unique
- Arrival time: must be unique
"""


def get_num_processes():
    while True:
        raw = input("Enter the no. of process: ").strip()
        if not raw.isdigit():
            print("Invalid input. Please enter a whole number.")
            continue
        n = int(raw)
        if n < 3 or n > 10:
            print("Invalid number of processes. Must be between 3 and
10.")
            continue
        return n


def get_unique_process_id(index, existing_ids):
    while True:
        pid = input(f"Enter process ID for Process{index}: ").strip()
        if pid == "":
            print("Process ID cannot be empty.")
            continue
        if pid in existing_ids:
```

```
            print(f"Process ID '{pid}' is already used. It must be
unique.")
            continue
        return pid


def get_unique_arrival_time(pid, existing_arrivals):
    while True:
        raw = input(f"Enter arrival time for {pid}: ").strip()
        if not raw.isdigit():
            print("Invalid input. Arrival time must be a non-negative
whole number.")
            continue
        at = int(raw)
        if at in existing_arrivals:
            print(f"Arrival time {at} is already used by another process.
It must be unique.")
            continue
        return at


def get_burst_time(pid):
    while True:
        raw = input(f"Enter burst time for {pid}: ").strip()
        if not raw.isdigit() or int(raw) <= 0:
            print("Invalid input. Burst time must be a positive whole
number.")
            continue
        return int(raw)


def compute_fcfs(processes):
    """
    processes: list of dicts with keys: pid, arrival, burst
    FCFS orders execution by arrival time (ties broken by input order).
    Returns the same list of dicts, each updated with:
        start, completion, turnaround, waiting
    """
    # Order of execution in FCFS = order of arrival
    ordered = sorted(processes, key=lambda p: p["arrival"])
```

```
    current_time = 0
    for p in ordered:
        # CPU may be idle if the next process hasn't arrived yet
        if current_time < p["arrival"]:
            current_time = p["arrival"]

        p["start"] = current_time
        p["completion"] = current_time + p["burst"]
        p["turnaround"] = p["completion"] - p["arrival"]
        p["waiting"] = p["turnaround"] - p["burst"]

        current_time = p["completion"]

    return ordered


def main():
    print("=== FCFS CPU Scheduling Simulator ===\n")

    n = get_num_processes()

    processes = []
    used_ids = set()
    used_arrivals = set()
    bursts = []
    pids = []
    arrivals = []

    for i in range(1, n + 1):
        pid = get_unique_process_id(i, used_ids)
        used_ids.add(pid)
        pids.append(pid)

    for i in range(0, n ):
        arrival = get_unique_arrival_time(pids[i], used_arrivals)
        used_arrivals.add(arrival)
        arrivals.append(arrival)

    for i in range(0, n):
```

```
        pburst = get_burst_time(pids[i])
        bursts.append(pburst)

    for i in range(0, n):
        processes.append({"pid": pids[i], "arrival": arrivals[i], "burst":
bursts[i] })

    results = compute_fcfs(processes)

    total_waiting = sum(p["waiting"] for p in results)
    total_turnaround = sum(p["turnaround"] for p in results)
    avg_waiting = total_waiting / n
    avg_turnaround = total_turnaround / n

    print(f"\nAverage waiting time: {avg_waiting:.1f}")
    print(f"Average turn-around time: {avg_turnaround:.1f}")


if __name__ == "__main__":
    main()
```

SAMPLE OUTPUT:

```
=== FCFS CPU Scheduling Simulator ===

Enter the no. of process: 13
Invalid number of processes. Must be between 3 and 10.
Enter the no. of process: f
Invalid input. Please enter a whole number.
Enter the no. of process: 5
Enter process ID for Process1: P1
Enter process ID for Process2: P1
Process ID 'P1' is already used. It must be unique.
Enter process ID for Process2: P2
Enter process ID for Process3: P3
Enter process ID for Process4: P4
Enter process ID for Process5: P5
Enter arrival time for P1: 2
Enter arrival time for P2: 2
Arrival time 2 is already used by another process. It must be unique.
Enter arrival time for P2: 0
Enter arrival time for P3: 3
Enter arrival time for P4: 9
Enter arrival time for P5: 6
Enter burst time for P1: 8
Enter burst time for P2: 4
Enter burst time for P3: 2
Enter burst time for P4: 5
Enter burst time for P5: 7

Average waiting time: 6.2
Average turn-around time: 11.4
PS C:\PROJECTS\MISC>
```

QUESTIONS:

1. How can you determine which CPU algorithm is effective?

The effectiveness of a CPU scheduling algorithm is determined by evaluating key performance metrics, including average waiting time, average turnaround time, CPU utilization, and throughput. An algorithm is considered more effective when it minimizes average waiting and turnaround time while maximizing CPU utilization and throughput. Fairness, particularly the avoidance of process starvation, is also a significant consideration.

2. How to compute the waiting time and burst time of every process?

The burst time of a process is a predefined input value representing the amount of CPU time required for its execution; it is not computed. Waiting time, however, is derived using the following formula:

Waiting Time = Turnaround Time − Burst Time

where Turnaround Time is calculated as:

Turnaround Time = Completion Time − Arrival Time

3. How to compute the average waiting time and average burst time of every process?

The average waiting time and average turnaround time are obtained by summing the respective values of all processes and dividing by the total number of processes:

Average Waiting Time = (Total Waiting Time of All Processes) ÷ (Number of Processes)

Average Turnaround Time = (Total Turnaround Time of All Processes) ÷ (Number of Processes)

CONCLUSION:

This program successfully simulates the FCFS CPU scheduling algorithm, computing each process's waiting time and turnaround time based on arrival order while enforcing proper input validation. The results demonstrate that although FCFS is simple to implement, it is not always efficient, as longer processes arriving early can delay shorter ones behind them (the convoy effect), leading to higher average waiting and turnaround times compared to other scheduling algorithms. This exercise reinforced key operating system concepts on process scheduling and its impact on overall system performance.